BCA60205 Data Warehousing and Data Mining

Classification and Regression

Module 3

Table of Contents

Section ID	Topic	Key Concepts
1.0.	Overview of classification &regression	Discrete vs Continuous prediction
2.0	Types of classification & regression in data mining	Overview of algorithm for the data mining
2.1	C4.5 Algorithm	Gain Ratio, Pruning, ID3 successor
2.2	CART Algorithm	Gini Index, Binary Splitting
3.1	Naive Bayes	Independence assumption, Bayes Theorem
3.2	Bayesian Networks	Probabilistic Graphical Models (DAG)
4.1	Classification Metrics	Accuracy, Precision, Recall, F1 Score
4.2	Regression Metrics	Mean Squared Error (MSE)
5.1	Bagging	Parallel models, Variance reduction, Random Forest
5.2	Boosting	Sequential models, Bias reduction, AdaBoost

1.0. Overview of classification and regression

Classification and regression are two fundamental **supervised learning techniques** used in data mining and machine learning to make predictions based on labeled data. **Classification** deals with predicting **categorical outcomes(discrete values)**, where the output belongs to a predefined class or label. The objective is to assign input data to one of the known categories. Common examples include spam detection (spam/not spam), disease diagnosis (yes/no), and student result prediction (pass/fail). Typical classification algorithms include Decision Trees, Naïve Bayes, Support Vector Machines, and Random Forests.

Regression, on the other hand, focuses on predicting **continuous numerical values**. It estimates the relationship between input variables and a numeric output. Examples include predicting house prices, sales forecasting, and temperature estimation. Common regression techniques include Linear Regression, Decision Tree Regression, and Regression-based ensemble models.

Feature	*Classification*	*Regression*
Output	Categorical	Continuous
Objective	Assign class labels	Predict numeric values
Example	Spam detection	House price prediction
Nature of Output	Discrete	Real-valued
Algorithms Used	Decision Tree, Naïve Bayes, SVM, Random Forest	Linear Regression, Decision Tree Regression
Evaluation Metrics	Accuracy, Precision, Recall, F1-score	MSE, RMSE, MAE
Decision Boundary	Exists (separates classes)	Not applicable
Use Case	Yes/No, True/False, Class labels	Forecasting, estimation
Error Measurement	Misclassification error	Prediction error (distance from actual value)

Table 1: key Difference between classification vs regression

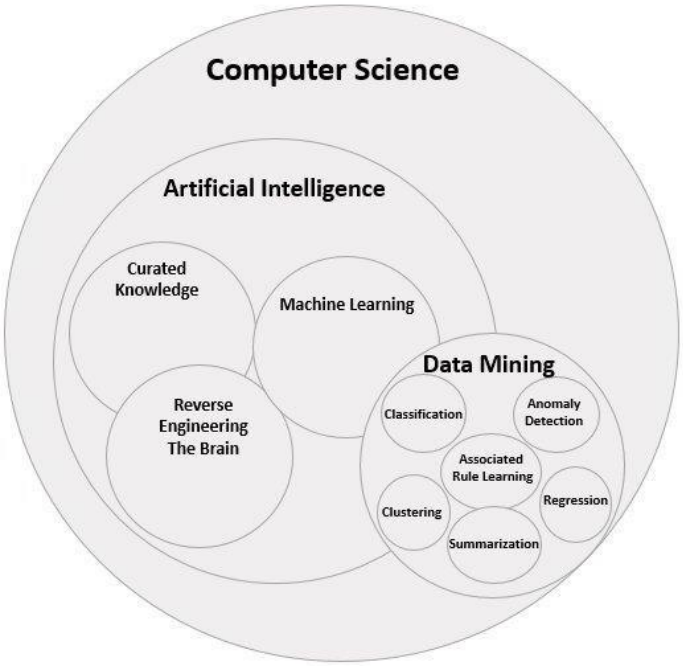

Figure 1

Classification in data mining is a supervised learning method that assigns labels to data points based on known examples. You provide an algorithm with labeled data, and it learns patterns that guide future predictions.

i) <u>Labeled Data:</u> You already know the correct labels for each example, so you use these labelled instances to train a classification model. The model grasps the underlying patterns, like how certain words might indicate spam or how specific behaviors imply higher customer churn.

ii) <u>Unseen Data:</u> You test the model with data that lacks predefined labels to see if the model can correctly predict categories. You validate its accuracy and adjust the model's parameters if the predictions miss the mark.

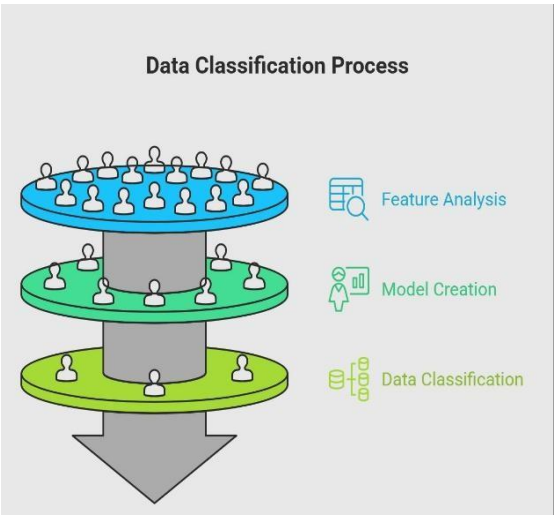

Figure 2

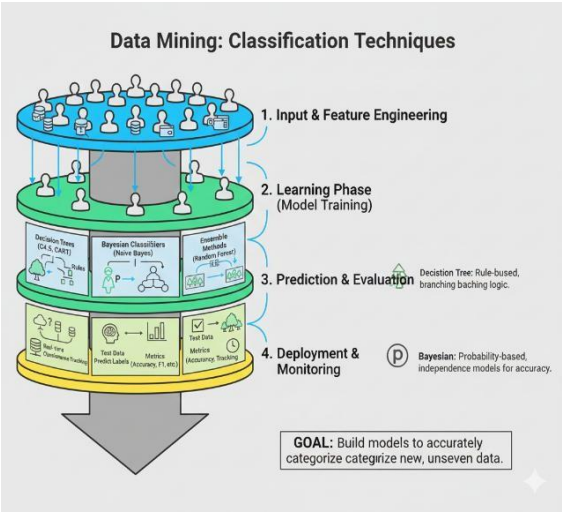

Figure 3

Why Is Classification Important for Organizations?

Classification meets that need by sorting through data and pinpointing valuable connections.t's so crucial for companies of all shapes, sizes, and domains.It Helps With risk management.By classifying transactions based on historical patterns, you can spot signs of suspicious transactions or unreliable clients.It Helps With customer engagement.You group individuals by their buying behavior or demographic details, then tailor campaigns or offers that resonate with each segment.It helps with eesource allocation.Once you know which classes require immediate attention, you distribute budget or manpower to the most pressing areas.

2.0 Types of classification in data minig

Some tasks require binary categories, while others involve multiple or overlapping labels. Each type offers unique advantages, so selecting the right one is crucial for effective analysis.

1. Binary Classification

Binary classification assigns **one of two possible class labels** to each data instance.
Mathematically, the target variable is: **$y \in \{0,1\}$**
The model learns a decision function: **$f(x) \rightarrow \{0,1\}$**
Examples:
Fraud Detection: y=1 (Fraud), y=0(Legitimate)
Virus Detection: Infected / Clean
Authentication: Allow / Deny
Key Point:
Binary classification is used when the outcome is a **yes/no** decision with minimal ambiguity.

2. Multi-class Classification

Multi-class classification assigns **exactly one label** from **three or more classes** to each instance.
Mathematically: $y \in \{c1, c2, c3, \ldots, ck\}, k>2$
The predicted class is: $\hat{y} = \underset{c_i}{\operatorname{argmax}} P(c_i|x)$

Examples:
Language detection: English / French / Spanish
Product categories: Electronics / Clothing / Appliances
Disease diagnosis: Disease A / B / C
Key Point: Each data point belongs to **only one class** among many.

3. Multi-label Classification

Multi-label classification allows **multiple labels** to be assigned to a **single instance**.
Mathematically: $y \subseteq \{c1, c2, \ldots, ck\}$
Each label is predicted independently: $f(x)=[y1, y2, \ldots, yk], yi \in \{0,1\}$
Examples:
Music genres: Rock + Indie + Alternative
News articles: Politics + Economics
Movies: Action + Comedy + Adventure
Key Point: Labels **can overlap**, and multiple classes may be true at once.

4. Nominal Classification

Nominal classification involves **unordered categorical labels** with no ranking.
Mathematically: $y \in \{c1, c2, c3, \ldots\}$ **(no order)**
Examples:
Pet types: Cat, Dog, Bird
Car brands: Toyota, BMW, Tesla
Payment modes: Cash, Card, Wallet
Key Point: Classes are **distinct but non-ranked**.

5. Ordinal Classification

Ordinal classification deals with **ordered categorical labels**, but **without measurable distance** between classes.
Mathematically:

$$C_1 \prec C_2 \prec C_3 \prec \cdots \prec C_k$$

Examples:
Ratings: Poor < Average < Good < Excellent
Education level: Bachelor < Master < PhD
Satisfaction levels: Low < Medium < High
Key Point: Classes have a **natural order**, but numerical differences are **not defined**.

****One-line memory rule:**
Binary = 2 classes, Multi-class = 1 of many, Multi-label = many at once, Nominal = no order, Ordinal = ordered labels.

Type of Classification	*Number of Classes*	*Label Nature*	*Key Characteristic*	*Examples*
Binary Classification	2	Categorical	Produces a yes/no decision	Spam vs Not Spam, Fraud vs Legitimate
Multi-class Classification	More than 2	Categorical	Each instance belongs to only one class	Language detection, Product categories
Multi-label Classification	Multiple at once	Categorical	An instance can belong to multiple classes	Music genres, News tagging
Nominal Classification	Any	Unordered categories	Categories have no inherent order	Pet types, Car brands
Ordinal Classification	Any	Ordered categories	Categories follow a logical order	Ratings, Education levels

Table 2 : Summary

□ Algorithms Are Commonly Used in Classification

Data is usually classified using two main approaches: generative and discriminative. Generative models learn the joint probability distribution of features and classes and then use this knowledge to predict unseen outcomes. Discriminative models focus on decision boundaries and learn how to map features to specific labels without modeling how the data is generated.Both strategies aim to find meaningful structure within the data but they tackle the task from different angles. Below, you'll see major classification algorithms organized by these ideas – generative and discriminative – along with practical examples.

1. Decision Tree Algorithm (Discriminative)

A decision tree splits data into subsets based on feature values to minimize impurity at each node. The splitting criterion is usually based on Information Gain or Gini Index.

Entropy , H(S) = - Σ (pi * log2(pi))
Gini Index: – Σ [p_i * log_2(p_i)]
Where:
H(S): The entropy of the set S.
Σ (Sigma): Represents the summation over all classes.
p_i: The probability or proportion of examples belonging to class i.
log_2: The logarithm to the base 2.
The feature that produces the maximum information gain or minimum Gini value is selected for splitting. **Examples:** Loan approval, medical diagnosis, customer segmentation.

2. Random Forest Algorithm (Discriminative)

Random Forest builds multiple decision trees using bootstrap sampling and random feature selection. Each tree votes, and the final class is chosen by majority voting.

Voting Rule: $(\hat{y})$ = Mode of $\{y_1, y_2, ..., y_n\}$
Where:
$\hat{y}$ (y-hat): Represents the final predicted class or result.
mode: Refers to the most frequent value in a set.
$y_1, y_2, ..., y_n$: Represent the individual predictions or "votes" from each tree in the forest.Statistically, ensemble averaging reduces variance, leading to better generalization.**Examples:** Fraud detection, recommendation systems, predictive maintenance.

3. Naive Bayes Algorithm (Generative)

Naive Bayes is a probabilistic classifier based on Bayes' Theorem, assuming conditional independence among features.

Bayes' Theorem: $P(C|X) = [P(X|C) * P(C)] / P(X)$
Where:
P(C|X): Probability of the **Class** given the **Data** (Posterior).
P(X|C): Probability of the **Data** given the **Class** (Likelihood).
P(C): Probability of the **Class** occurring (Prior).
P(X): Total probability of the **Data** (Evidence).
Despite the "naive" independence assumption, it performs exceptionally well on high-dimensional data such as text.**Examples:** Spam filtering, news classification, sentiment analysis.

4. Logistic Regression Algorithm (Discriminative)

Logistic regression models the probability of a binary outcome using the sigmoid function.

Sigmoid Function: $P(y=1|x) = 1 / (1 + e^{-z})$ where $z = w^T * x$.
Where
P(y=1|x): The predicted probability that the data point belongs to class 1.
e: Euler's number (approximately 2.718).
z: The "Logit" or linear combination of weights (w) and inputs (x).
$w^T x$: The dot product of the weights and the input features. Classification is performed by applying a threshold (usually **0.5**) on predicted probabilities. **Examples:** Churn prediction, disease risk assessment, marketing response prediction.

5. Support Vector Machine (Discriminative)

SVM finds the optimal hyperplane that maximizes the margin (distance) between different classes.

Decision Boundary: $w^T * x + b = 0$

Where:
w^T: The Transpose of the Weight vector (determines the orientation of the boundary).
x: The Input Feature vector.
b: The Bias term (determines the distance of the boundary from the origin).

Margin Maximization:

Kernel functions (like RBF or Polynomial) enable non-linear separation in higher dimensions
min 1/2 * ||w||^2.
Where:
min: Stands for minimization (the goal is to find the smallest possible value).
1/2: A mathematical constant used to simplify the derivative during optimization.
||w||: Represents the "norm" or magnitude (length) of the weight vector.
^2: Indicates that the magnitude is squared.**Examples:** Handwritten digit recognition, bioinformatics, email classification.

6. k-Nearest Neighbors (Discriminative)

k-NN is a "lazy learner" that classifies data points based on distance measures, most commonly Euclidean distance.
Euclidean Distance: d(p, q) = sqrt(sum((pi - qi)^2))
Where:
d(p, q): The distance between two points, p and q.
sqrt: The square root function.
Σ (Sum): The summation of values for all dimensions (i).
p_i and q_i: The coordinates of points p and q in the ith dimension.
^2: Indicates that the difference is squared to ensure the distance is positive.
The predicted class is selected using majority voting among the k nearest neighbors.Examples: Recommendation systems, document retrieval, content moderation.

7. Neural Networks (Discriminative / Hybrid)

Neural networks consist of input, hidden, and output layers of neurons performing weighted sums followed by activation functions.
Neuron Computation: y = f(wT * x + b).
Where:
y: The output of the neuron (the activation value).
f: The activation function (e.g., ReLU, Sigmoid, or Tanh) that introduces non-linearity.
wT: The Transpose of the Weight vector.
x: The Input Feature vector.
b: The Bias term, which allows the activation function to shift.
Training is performed using **backpropagation** and gradient descent to minimize a loss function such as cross-entropy. **Examples:** Image recognition, speech processing, fraud detection.

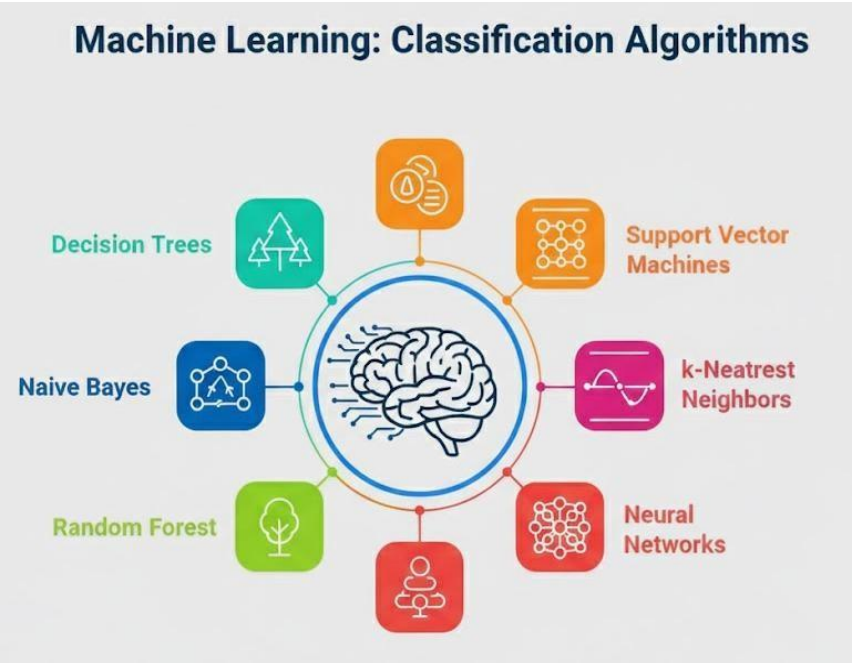

Figure 4

Build a Classification Model Step-by-Step

Building a strong classification model involves a series of clear stages, each addressing key challenges like data collection and model testing. Understanding basic mathematical notations helps in tuning parameters and interpreting results, even without advanced math knowledge.Here are the clean bulleted steps for the Process of Building a Classification Model:

1. **Data Collection** – Gather relevant and labeled data from various sources.
2. **Data Preprocessing** – Clean data by handling missing values, noise, and inconsistencies.
3. **Feature Selection & Engineering** – Choose important features and transform them to improve model performance.
4. **Model Selection** – Select an appropriate classification algorithm based on the problem and data.
5. **Training & Validation** – Train the model on training data and validate it using unseen data.
6. **Model Evaluation** – Assess model performance using metrics such as accuracy, precision, recall, and F1-score.
7. **Model Deployment & Monitoring** – Deploy the model in real-world applications and monitor its performance over time.

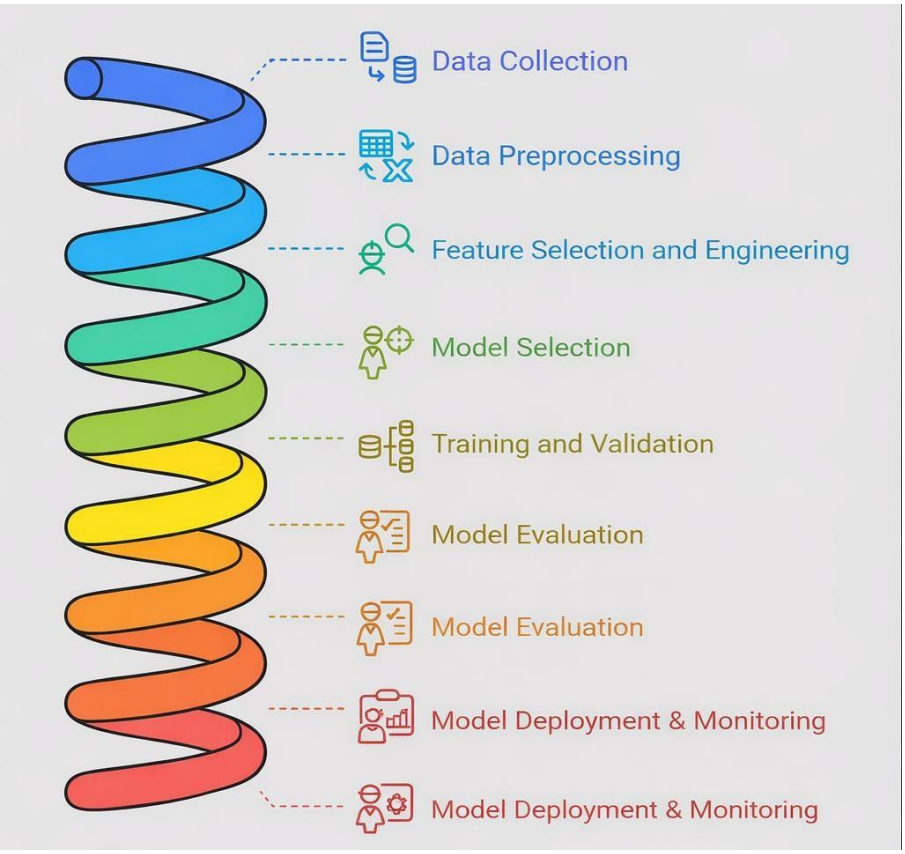

Figure 5

Table 3: Summary Table for Quick Reference

How to handle the data quality and dataset issues?

Data quality issues must be addressed **before model training** to ensure reliable and accurate results. Common problems and their solutions are outlined below:

1. Missing Data

Missing values reduce model accuracy and bias results.
Handling methods:

1. Remove rows or columns with excessive missing values.
2. Replace missing values using **mean, median, or mode.**
3. Use advanced techniques like **interpolation or predictive imputation.**

2. Noisy and Inconsistent Data

Noise includes incorrect, duplicate, or contradictory records.
Handling methods:

1. Detect and remove duplicate entries.
2. Apply smoothing techniques.
3. Use validation rules to correct inconsistent values.

3. Outliers

Outliers distort statistical measures and model behavior.
Handling methods:

1. Identify outliers using **Z-score** or **IQR.**
2. Remove or cap extreme values.
3. Apply robust scaling techniques.

4. Imbalanced Dataset

Occurs when one class dominates others.
Handling methods:

1. Oversampling minority class (e.g., SMOTE).
2. Undersampling majority class.
3. Use class-weighted algorithms.

5. Inconsistent Data Formats

Different units or formats create confusion.
Handling methods:

1. Standardize units and naming conventions.
2. Convert categorical variables into numeric form.
3. Normalize or standardize numerical features.

6. Irrelevant or Redundant Features

Unnecessary features increase complexity and overfitting.
Handling methods:

1. Feature selection using correlation analysis.
2. Remove low-variance features.
3. Apply dimensionality reduction (PCA).

7. Data Leakage

Occurs when training data contains information unavailable during prediction.
Handling methods:

1. Separate training and test data properly.
2. Apply preprocessing **after** data splitting.
3. Avoid using future information.

Figure 6

Challenges in Data Quality and Dataset Management with Solutions

Challenge	*How to Address?*
Imbalanced Classes	▪ Oversample the minority class (e.g., SMOTE) ▪ Under sample the majority class when appropriate. ▪ Adjust class weights in algorithms
Missing Values	▪ Impute numerical values using mean or median. ▪ Remove rows only when data cannot be recovered
Outliers or Noise	▪ Detect anomalies using Z-score or Interquartile Range (IQR). ▪ Verify whether outliers are genuine rare cases or data errors
Overfitting and Underfitting	▪ Apply cross-validation to evaluate generalization. ▪ Use regularization techniques or early stopping.
Large or Complex Datasets	▪ Split data into manageable chunks or use distributed computing. ▪ Monitor memory usage and processing time. ▪ Apply dimensionality reduction techniques

Table 4

Tools and Technologies Commonly Used for Classification

Classification in data mining relies on programming languages, data mining tools, and libraries that simplify model development and improve efficiency.

1. Programming Languages

1. **Python**: Most popular for classification due to simplicity and rich libraries like *Scikit-learn* for logistic regression, decision trees, SVM, etc.
2. **R**: Strong in statistics and visualization; widely used in research with packages like *caret* for classification workflows.

2. Data Mining Tools

1. **RapidMiner**: Drag-and-drop interface; commonly used in finance for fraud detection and risk classification.
2. **KNIME**: Open-source, modular platform; used in telecom and business analytics for customer classification.
3. **WEKA**: Java-based tool with built-in classifiers like Naive Bayes and Random Forest; popular in academics.

3. Libraries and Frameworks

1. **Scikit-learn**: Easy-to-use Python library supporting SVM, KNN, Decision Trees, and Random Forests.
2. **TensorFlow & Keras**: Used for deep learning-based classification such as image and speech recognition.
3. **PyTorch**: Flexible deep learning framework preferred for research and complex neural network models.

Regression in data mining is a statistical technique that is used to model the relationship between a dependent variable and one or more independent variables. The goal is to predict the value of the dependent variable based on the values of the independent variables. The dependent variable is also called the response variable, while the independent variable(s) is also known as the predictor(s).

For example, let's say we want to predict the price of a house based on its size, number of bedrooms, and location. In this case, price is the dependent variable, while size, number of bedrooms, and location are the independent variables. By analyzing the historical data of houses with similar characteristics, we can build a regression model that predicts the price of a new house based on its size, number of bedrooms, and location.

There are several types of regression models, including linear regression, logistic regression, and polynomial regression. Linear regression in data mining is the most commonly used type, which assumes a linear relationship between the independent and dependent variables. However, nonlinear relationships may exist between the variables in some cases, which can be captured using nonlinear regression models.

Types of Regression Techniques

There are several types of techniques used for regression in data mining and statistical analysis. Some of the most commonly used regression techniques are mentioned below -

Linear Regression

Linear regression in data mining is a statistical technique used to model the relationship between a dependent variable and one or more independent variables, assuming a linear relationship between them. The goal is to find the best-fit line that minimizes the distance between the observed and predicted values.Linear Regression Formula **y = b0 + b1 * x + e.**

Where

- y: The dependent variable (the outcome you are trying to predict).
- x: The independent variable (the predictor or input).
- b0: The intercept (the value of y when x is zero).
- b1: The slope (the amount y changes for every one-unit increase in x).
- e: The error term (the difference between the actual value and the predicted value).

The parameters of the linear regression model are estimated using the least squares method, which minimizes the sum of the squared differences between the observed and predicted values. Linear regression in data mining can also be extended to multiple linear regression, where there are multiple independent variables.

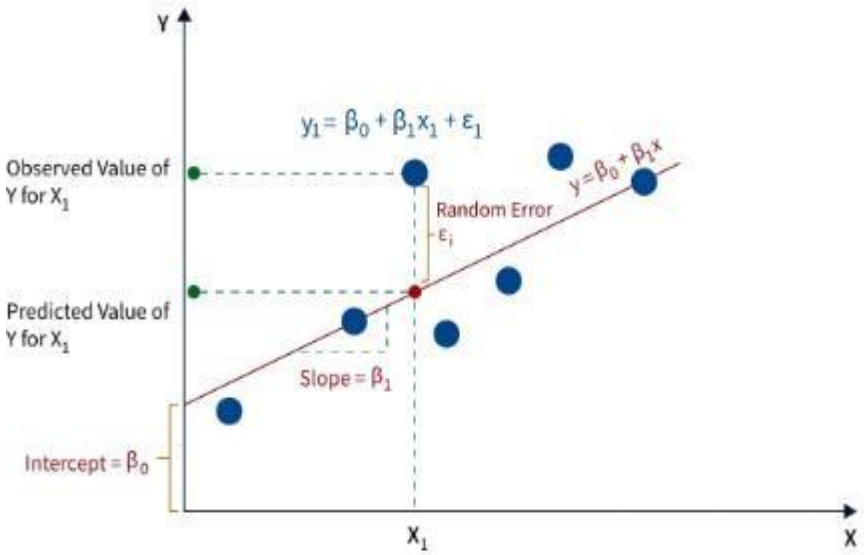

Figure 7

Logistic Regression

Logistic regression is a statistical technique used in data mining to model the relationship between categorical (usually binary) dependent variables and one or more independent variables. Instead of predicting exact values, it predicts probabilities that an instance belongs to a particular class.

It uses the logistic (sigmoid) function to convert a linear combination of input features into a value between 0 and 1, which represents class probability.

The logistic regression model uses the logistic function to model the relationship between the independent variables and the dependent variable. The logistic function transforms the output of the linear regression model to a value between 0 and 1, representing the probability of the dependent variable taking a particular value.
The equation of a logistic regression model is p = 1 / (1 + exp(−z)),p is the probability of the dependent variable taking a particular value, z is the linear combination of the independent variables and their coefficients, and exp is the exponential function. The parameters of the logistic regression model are estimated using the maximum likelihood method, which maximizes the likelihood of the observed data given the model.

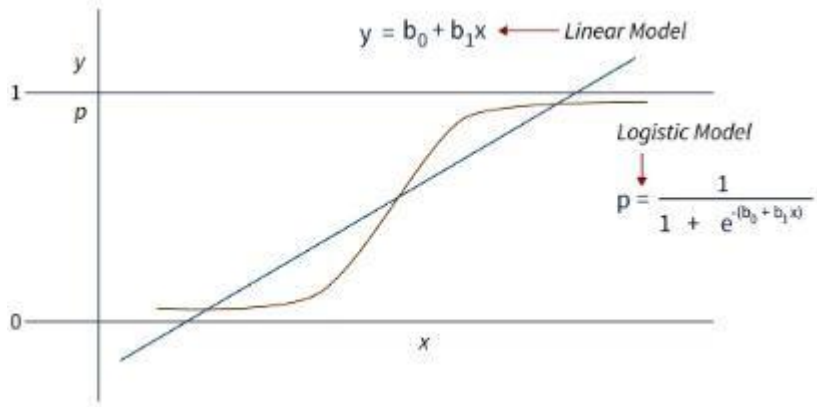

Figure 8

<u>Polynomial Regression</u>
Polynomial regression in data mining is a statistical technique used to model the relationship between a dependent variable and one or more independent variables, assuming a polynomial relationship between them. In polynomial regression, the relationship between the independent and dependent variables is modelled as an nth-degree polynomial function. Polynomial regression in data mining is useful when the relationship between the independent and dependent variables is nonlinear, but a linear model is inadequate.
The equation of a polynomial regression model is – $\mathbf{y = b0 + b1x + b2x^2 + ... + bn \cdot x^n + e}$
Where:

- **y** is the dependent variable.
- **x** is the independent variable.
- **b0, b1, b2, ..., bn** are the coefficients of the polynomial.
- **n** is the degree of the polynomial.
- **e** is the error term.

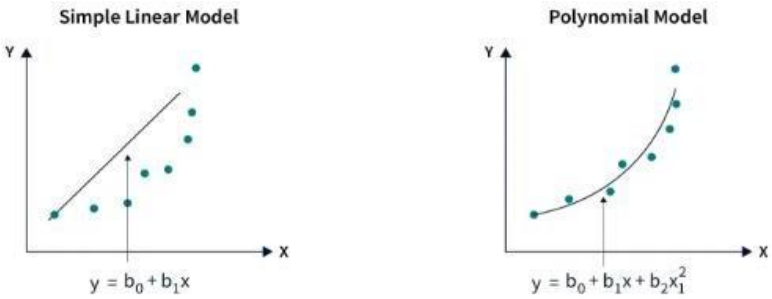

Figure 9

<u>Lasso Regression</u> (L1 regularization)

Lasso regression in data mining is a linear regression technique used for feature selection and regularization by adding a penalty term to the cost function. The penalty term is the L1 norm of the coefficients, which shrinks the coefficients towards zero and can result in some of them being exactly zero, effectively performing feature selection. The L1 norm of coefficients refers to the sum of the absolute values of the regression coefficients. The L1 norm is also known as the Manhattan norm. Lasso regression is useful when there are many independent variables, some of which are irrelevant or redundant. Lasso adds a penalty equal to the **absolute value** of the magnitude of the coefficients. Because of the geometry of the absolute value function, it can force some coefficients to be exactly zero.

<u>The Cost Function</u>:

$$\min_{w}\left(\sum_{i=1}^{n}(y_i-\sum_{j=1}^{p}x_{ij}w_j)^2+\lambda\sum_{j=1}^{p}|w_j|\right)$$

Penalty term :
$\lambda \Sigma |w_j|$ (also written as $\lambda ||w||_1$)
λ: The Greek letter Lambda (tuning parameter).
Σ: Summation symbol.
$|w_j|$: Absolute value of the weight/coefficient.
$||w||_1$: The L1 Norm notation.

Behavior : Lasso performs **feature selection**. If a variable is not contributing significantly, Lasso will set its weight to zero, effectively removing it from the model.

<u>Ridge Regression</u>(L2 regularization)

Ridge regression in data mining is a linear regression technique used for regularization by adding a penalty term to the cost function. The penalty term is the L2 norm of the coefficients, which shrinks the coefficients towards zero but does not result in exact zeros, unlike Lasso regression. The L2 norm of coefficients refers to the square root of the sum of the squared values of the regression coefficients. The L2 norm is also known as the Euclidean norm. Ridge regression is useful when there is multicollinearity among the independent variables, which can lead to unstable and unreliable coefficient estimates.

Ridge regression adds a penalty equal to the **square of the magnitude** of coefficients. This discourages the weights from becoming too large but never forces them to exactly zero.

The Cost Function:

$$\min_{w} \left(\sum_{i=1}^{n} (y_i - \sum_{j=1}^{p} x_{ij} w_j)^2 + \lambda \sum_{j=1}^{p} w_j^2 \right)$$

Penalty term :
$\lambda \Sigma |w_j|$ (also written as $\lambda ||w||_1$)
λ: The Greek letter Lambda (tuning parameter).
Σ: Summation symbol.
$|w_j|$: Absolute value of the weight/coefficient.
$||w||_1$: The L1 Norm notation.

Behavior: As λ, increases the coefficients shrink toward zero asymptotically. This is ideal when you have many small effects and want to prevent overfitting.

Elastic Net

Elastic Net is the "best of both worlds" approach. It is particularly useful when:

1. **High Correlation:** You have multiple features that are highly correlated with each other. Lasso might randomly pick one, but Elastic Net will keep them in the model (like Ridge) while still allowing some to go to zero (like Lasso).
2. **p > n:** You have more features (p) than observations (n). Lasso can only select at most n variables, but Elastic Net can overcome this limitation.

Comparison Table

Feature	*Ridge (L2)*	*Lasso (L1)*	*Elastic Net (L1+L2)*
Constraint Shape	Circle	Diamond	"Rounded" Diamond
Feature Selection	No	Yes	Yes
Handling Collinearity	Good (Shrinks together)	Poor (Picks one)	Excellent (Groups them)
Model Sparsity	Dense (Low)	Sparse (High)	Variable (Adjustable)

Table 5

Choosing the Right Model

Scenario	Recommended Model
Many features, most are important	**Ridge**
Many features, only a few are important	**Lasso**
Highly correlated features (Collinearity)	**Elastic Net** (or Ridge)
Need a simple, interpretable model	**Lasso**
More features than data points (p > n)	**Elastic Net**

Table 6

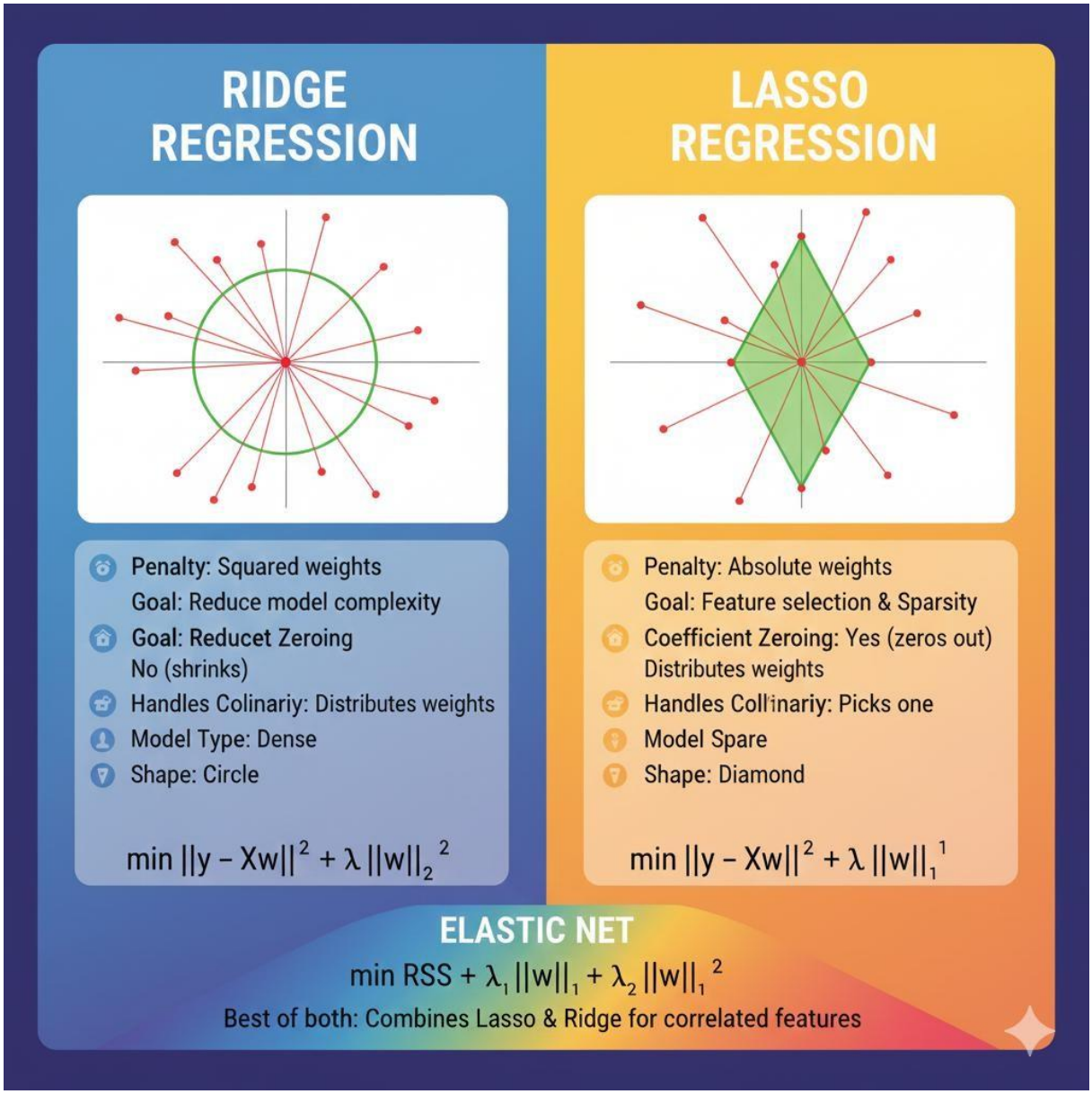

Figure 10

Decision tree algorithms like C4.5 and CART (Classification and Regression Trees) are the "founding fathers" of modern tree-based machine learning. While they both use a "divide and conquer" approach to split data into subsets, they differ significantly in their math, tree structure, and pruning techniques.

2.1. C4.5 Algorithm (The Successor to ID3)

Developed by Ross Quinlan, C4.5 is an improvement over the earlier ID3 algorithm. It is primarily used for **classification** tasks.

1. **Splitting Criterion:** It uses **Gain Ratio**, which is an extension of Information Gain (based on Entropy). C4.5 introduces Gain Ratio to penalize attributes with many

possible values (like a "Date" or "ID" column) that would otherwise look like perfect splitters but offer no real predictive power.

2. **Tree Structure:** It can create **multi-way splits**. If a feature has three categories (Low, Medium, High), C4.5 can split the node into three distinct branches at once.
3. **Pruning:** It uses **Error-Based Pruning** (EBP). This is a "top-down" post-pruning approach that replaces branches with leaves if the estimated error rate of the leaf is lower than that of the branch.

Key Features:

- Handles both continuous and discrete data.
- Handles missing values by ignoring them in gain calculations or distributing the records proportionally.

Mathematical equation:

$$Entropy(S) = -\sum(i = 1)^{c} p_i log_2(p_i)$$

Where p_i is the probability of a class appearing in the dataset.

2.2 CART Algorithm (Classification and Regression Trees)

Developed by Breiman et al., CART is a more versatile algorithm used for both **classification** (predicting labels) and **regression** (predicting continuous numbers).

1. Splitting Criterion:

1. For Classification: Uses the Gini Impurity index. This measures how often a randomly chosen element from the set would be incorrectly labeled.
2. For Regression: Uses Least Squares Deviation (minimizing the squared error within the nodes).

2. Tree Structure: It only produces **binary trees**. Every internal node is strictly split into exactly two child nodes (a "Yes/No" or "True/False" logic), regardless of how many categories a feature has.

3. Pruning: It uses **Cost-Complexity Pruning**. This method builds a massive, overfitted tree first and then prunes back the branches that contribute the least to the overall model accuracy relative to the tree's complexity.

Key Features:

- Strictly binary in nature.
- Handles outliers well in regression tasks.

Mathematical equation:

$$Gini(S) = 1 - \sum(i = 1)^{c} p_i^2$$

- **C4.5** uses entropy-based Gain Ratio to create multi-way splits for classification, handling missing values and continuous data through error-based pruning.
- **CART** generates strictly binary trees for both classification (using Gini Impurity) and regression, utilizing cost-complexity pruning to optimize model size.

Comparison table

Feature	*C4.5*	*CART*
Output Type	Classification only	Classification & Regression
Tree Structure	Multi-way or Binary	Strictly Binary
Splitting Metric	Gain Ratio (Entropy-based)	Gini Impurity or Squared Error
Pruning Technique	Error-Based Pruning (EBP)	Cost-Complexity Pruning
Handling Continuous Data	Binning/Thresholding	Binary splits on thresholds
Developer	Ross Quinlan	Breiman, et al.

Table 7

Classifiers like **Naive Bayes** and **Bayesian Networks** are essential because they handle uncertainty and "prior knowledge" better than almost any other algorithm. While a Decision Tree or a Linear Regression looks for a "hard line" to separate data, Bayesian models look for the "most likely" explanation.

3.1 Naive Bayes Classifier

The "Naive" Bayes classifier is called such because it makes a massive simplifying assumption: **Conditional Independence**. It assumes that the presence of one particular feature in a class is unrelated to the presence of any other feature.**The mathematical equation :**

$$P(C|x_1, \dots, x_n) = (P(C) \text{ G } P(x_i|C)) / P(x_1, \dots, x_n)$$

Where:

- **P(C|X) (Posterior):** Probability of class C given features X.
- **P(X|C) (Likelihood):** Probability of features X given class C.
- **P(C) (Prior):** Initial probability of the class.
- **P(X) (Evidence):** Probability of the features occurring.

Types of Naive Bayes

- **Gaussian:** Assumes features follow a normal distribution (best for continuous data).
- **Multinomial:** Used for discrete counts (e.g., word counts in text classification).
- **Bernoulli:** Used for binary/boolean features (e.g., word exists or doesn't).

Why the "Naive" Assumption Matters?

In a standard Bayesian model, calculating the likelihood of many features together is mathematically difficult. By assuming independence, we can simplify the calculation into a product of individual probabilities:

$$P(X|C) = \text{G}(i = 1)^n P(x_i|C)$$

This simplification is what makes the algorithm **computationally fast** and allows it to work with a very small amount of training data compared to more complex models.

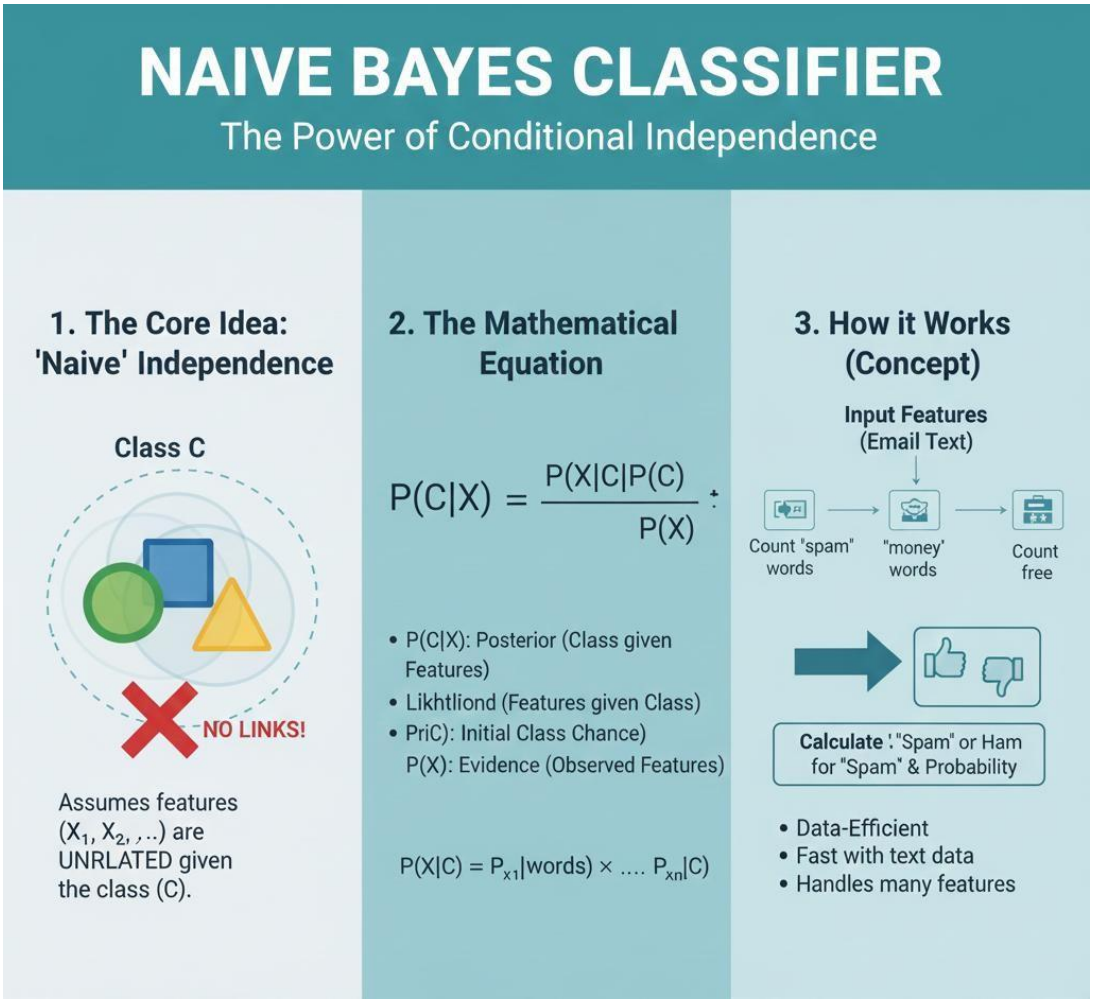

Figure 11

3.2 Bayesian Networks (Belief Networks)

Unlike Naive Bayes, **Bayesian Networks** do not assume all features are independent. Instead, they use a **Directed Acyclic Graph (DAG)** to represent the specific dependencies between variables.

Key Components :

- **Nodes:** Represent random variables (features or classes).
- **Edges (Arrows):** Represent direct dependencies (causal relationships).
- **Conditional Probability Tables (CPTs):** Each node has a table showing the probability of its values given the values of its "parents" in the graph.

The Mathematical Equation

A Bayesian Network represents the joint probability distribution of a set of variables **{X_1, ..., X_n}** as:

$$P(X_1, X_2, \dots, X_n) = \mathrm{G}(i = 1)^n P(X_i | Parents(X_i))$$

Why this formula is used?

In a **Bayesian Network**, we don't assume every feature is independent (like we do in Naive Bayes). Instead, we only assume a variable is dependent on its **"Parents"** (the nodes that have arrows pointing directly to it in the graph).

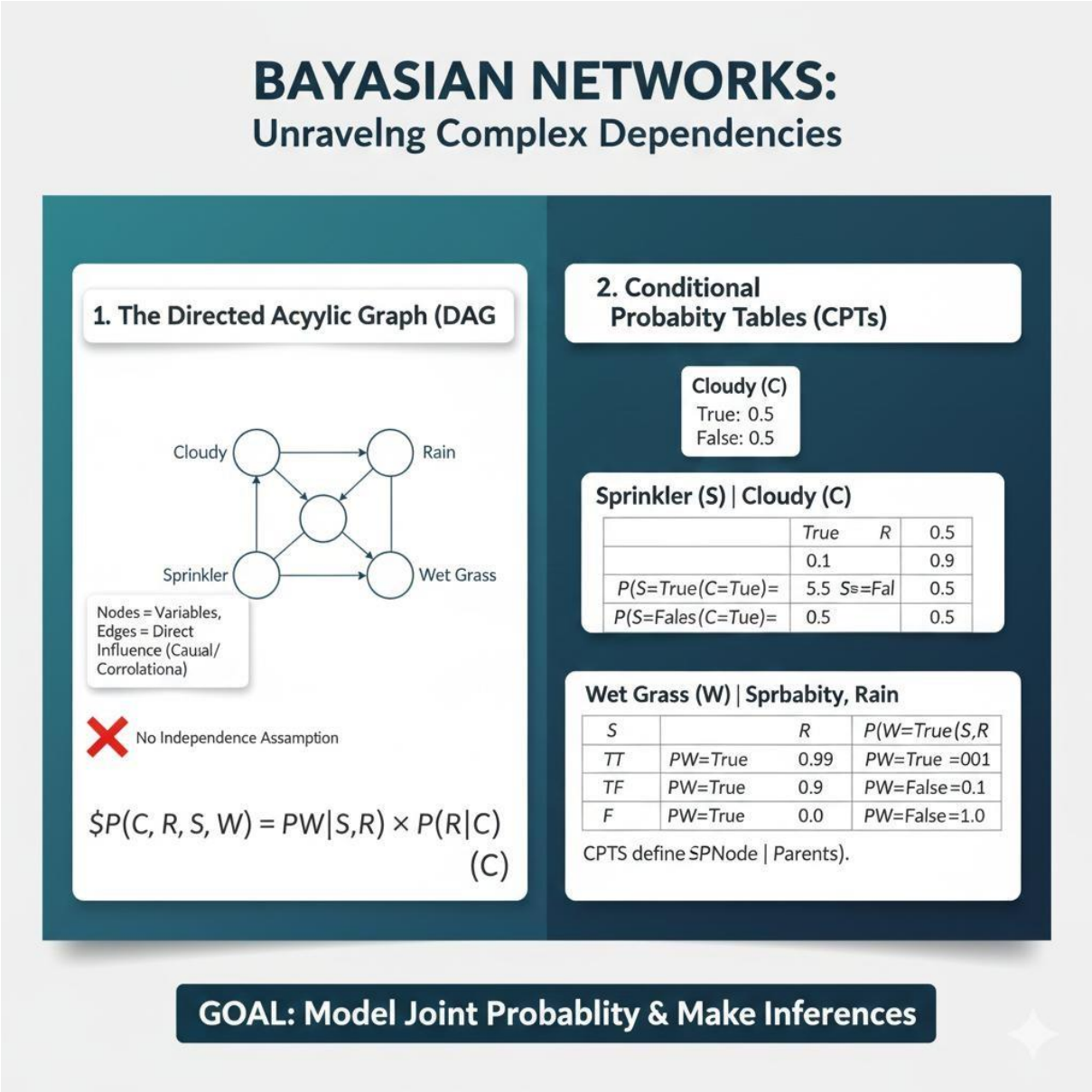

Figure 12

To help you understand how these classifiers work in practice, here is a classic mathematical word problem (The "Play Tennis" Problem) that demonstrates the step-by-step calculation for both Naive Bayes and a Bayesian Network.

1. Naive Bayes Mathematical Problem

The Scenario: You want to predict if a match will be played (**Yes/No**) based on the **Weather** (Sunny, Overcast, Rainy).

Dataset (History):

- Total Matches: 10
- Matches Played (**Yes**): 6
- Matches Canceled (**No**): 4

Likelihoods:

- If Yes: 2 were Sunny, 3 were Overcast, 1 was Rainy.
- If No: 3 were Sunny, 0 were Overcast, 1 was Rainy.

Problem: If today is **Sunny**, will the match be played?

The Step-by-Step Calculation:

1. **Calculate Priors:**
   - P(Yes) = 6/10 = 0.6
   - P(No) = 4/10 = 0.4
2. **Calculate Likelihoods (for Sunny):**
   - P(Sunny|Yes) = 2/6 = 0.33
   - P(Sunny|No) = 3/4 = 0.75
3. **Apply Bayes Theorem (Posterior):**

   - **Score for Yes:** P(Yes) × P(Sunny|Yes) = 0.6 × 0.33 = **0.198**
   - **Score for No:** P(No) × P(Sunny|No) = 0.4 × 0.75 = **0.300**

Conclusion: Since 0.300 > 0.198, the Naive Bayes classifier predicts **"No"** (Match cancelled).

2. Bayesian Network Mathematical Problem

The Scenario: A Bayesian Network represents dependencies. Let's look at a **Burglar Alarm** system.

- **B (Burglary):** Probability is 0.001.
- **A (Alarm):** Depends on Burglary.
- **M (Neighbour Mary Calls):** Depends on the Alarm.

The Logic: Instead of assuming Mary's calling and a Burglary are independent, we use the chain rule:

```
P(B, A, M) = P(B) * P(A|B) * P(M|A)
```

Problem: What is the probability that a Burglary happened (B), the Alarm went off (A), and Mary called (M)?

Given Probabilities:

- P(B) = 0.001
- P(A|B) = 0.95(95% chance alarm rings if there is a burglary)
- P(M|A) = 0.70(70% chance Mary hears the alarm and calls)

The Calculation: 0.001 × 0.95 × 0.70 = **0.000665**

Conclusion: The joint probability of all three events occurring is **0.0665%**.

Key Takeaway for your Notes:

- **In Naive Bayes:** We multiplied the weather by the class probability directly.
- **In Bayesian Networks:** We followed the "path" of the arrows ($Burglary \rightarrow Alarm \rightarrow Mary$)=P(B) × P(A|B) × P(M|A)to calculate the final probability.

Classification models are evaluated using **confusion matrix-based metrics**.Regression models are evaluated using **error-based metrics**.

4.1 Classification model evaluation metrics

Creating a powerful classification model is just the start; measuring its accuracy and reliability is crucial. Evaluation metrics show how well the model assigns labels, highlights errors, and balances false positives and negatives. To evaluate the performance of a classification model, we use specific mathematical metrics derived from the **Confusion Matrix**. This matrix serves as the foundation for all subsequent calculations.

1. The Foundation: Confusion Matrix

The Confusion Matrix is a table that categorizes predictions against actual outcomes.
TP (True Positive): Predicted 1, Actual 1
TN (True Negative): Predicted 0, Actual 0
FP (False Positive): Predicted 1, Actual 0 (Type I Error)
FN (False Negative): Predicted 0, Actual 1 (Type II Error)

2. Core Performance Metrics

Accuracy

The ratio of correct predictions to the total number of cases. It is most effective when classes are balanced.
Accuracy = {TP + TN}/{TP + TN + FP + FN}

Precision (Positive Predictive Value)

Measures the quality of positive predictions. It answers: "Of all instances predicted as positive, how many were actually positive?"
Precision = {TP}/{TP + FP}

Recall (Sensitivity)

Measures the ability to find all positive instances. It answers: "Of all actual positive instances, how many did we identify?"Recall = {TP}/{TP + FN}

F1-Score

The harmonic mean of Precision and Recall. It is used when you need a balance between the two, especially if there is an uneven class distribution.
F1 = 2 * (Precision * Recall) / (Precision + Recall)

3. Threshold-Based Curves

ROC-AUC (Receiver Operating Characteristic)

The ROC curve plots the **True Positive Rate (Recall)** against the **False Positive Rate (FPR)** at various threshold settings.

1. **FPR Calculation:** FPR = {FP}/{FP + TN}
2. **AUC (Area Under the Curve):** A single scalar value (0 to 1) representing the probability that the model ranks a random positive example higher than a random negative one.

Precision-Recall (PR) Curve

This plots Precision vs. Recall. It is more informative than ROC-AUC when dealing with **imbalanced datasets** (where the positive class is rare).

Metric	*Mathematical Formula*	*Focus*
Accuracy	(TP+TN) / Total	Overall correctness
Precision	TP / (TP+FP)	Accuracy of positive guesses
Recall	TP / (TP+FN)	Ability to catch all positives
F1-Score	F1 = 2 * (P * R) / (P + R)	Balanced trade-off

Table 8

4.2 Regression model evaluation metrics

Regression models predict a **continuous numerical value** (e.g., house price, temperature, sales).

Let:

- y_i is the actual value.
- $\hat{y}_i$ (y-hat subscript i) is the predicted value.
- **n** is the number of observations.

1. Mean Squared Error (MSE):

Mean Squared Error measures the average of the squared differences between the actual values and the predicted values.

Mathematical Formula:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Interpretation

- Lower MSE ⇒ better model.
- Penalizes **large errors more heavily.**

Where MSE Is Used :

1. Regression Model Evaluation

MSE is used to evaluate **regression models** such as: Linear Regression,Polynomial Regression,Multiple Linear Regression,Ridge and Lasso RegressionIt measures how close the predicted values are to the actual values.

2. Loss Function During Model Training

MSE is commonly used as a **loss (cost) function** in: Linear Regression optimization, Neural Networks (for regression tasks), Gradient Descent–based learning. The model parameters are updated to **minimize the MSE**.

3. Time Series Forecasting

MSE is used to evaluate forecasting accuracy in: Sales prediction, Weather forecasting, and Stock price prediction.

4. Model Comparison

When comparing multiple regression models on the same dataset, The model with **lower MSE** is considered better.

5. Signal Processing and Image Processing

MSE is used to measure: Signal reconstruction error, Image compression quality (difference between original and reconstructed image).

2. Root Mean Squared Error (RMSE)

RMSE is the **square root of MSE**, bringing the error back to the original unit.

Mathematical Formula

$$\text{RMSE} = \sqrt{\frac{1}{\downarrow}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Interpretation

- Easier to interpret than MSE.
- Sensitive to outliers.

3. Mean Absolute Error (MAE)

MAE measures the **average absolute difference** between actual and predicted values.

Mathematical Formula

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

Interpretation

- Less sensitive to outliers.
- Treats all errors equally.

4. Coefficient of Determination (R^2)

R^2 presents the **proportion of variance in the dependent variable** explained by the model.

Mathematical Formula

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$$

Interpretation

R^2 =1 → perfect fit

R^2 = 0 → no explanatory power

Can be **negative** for poor models

5. Adjusted (R^2)

Adjusted penalizes unnecessary predictors.

Mathematical Formula

$$\text{Adjusted } R^2 = 1 - \left(\frac{1 - R^2}{n - p - 1}\right)(n - 1)$$

Inerpretation

- Preferred for **multiple regression.**
- Increases only if new feature improves the model.

Regression models are evaluated using error-based metrics such as MSE, RMSE, and MAE, and goodness-of-fit metrics like R^2 and Adjusted R^2 .

Ensemble Methods

Ensemble methods combine **multiple weak learners** to build a **strong predictive model**, improving **accuracy, stability, and generalization**. Multiple machine learning models are combined to obtain a more accurate model. Stacking, bagging, and boosting are the three most popular **ensemble learning** techniques. Let: $h_1(x), h_2(x), \ldots, h_M(x)$be base learners the final prediction = aggregation of individual predictions.

Bias

Bias is the error introduced by making **simplifying assumptions** in a model.
A high-bias model is too simple and **fails to capture the true pattern** in the data, leading to **underfitting**.

Variance

Variance is the error caused by a model being **too sensitive to the training data**.
A high-variance model learns noise and fluctuations in the data, leading to **overfitting** and poor performance on new data.

Mathematical View

Total Error=Bias^2 + Variance + Irreducible Error Bias–
Variance Tradeoff

- Bagging → reduces variance
- Boosting → reduces bias
- Random Forest → reduces variance + correlation

Mathematical Framework

Let,

Training Data Definition $D = (x_i, y_i)^n{}_{(i=1)}$, Base learners $h_1(x), h_2(x), \dots, h_M(x)$.
The ensemble prediction for the ***regression*** and ***classification*** will be :
$\hat{y} = 1/M \sum(m = 1)^M h_m(x)$ and $\hat{y} = argmax_c \sum(m = 1)^M I(h_m(x) = c)$

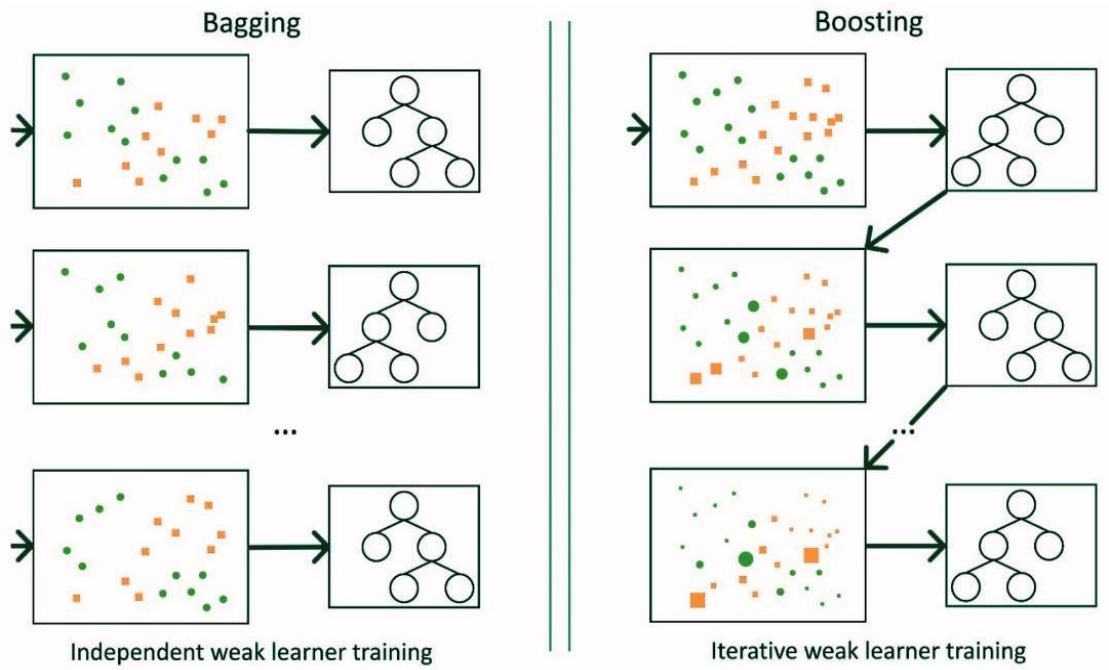

Figure 13

5.1 Bagging (Bootstrap Aggregating)

Bagging, short for **Bootstrap Aggregating**, is an ensemble method designed to reduce the variance of machine learning models. Bagging involves training multiple independent models on different subsets of the dataset and then combining their predictions to produce a final output. This method helps improve model performance by reducing overfitting and creating a more stable model.

How Bagging Works?

Bagging works by generating multiple bootstrapped datasets (random samples drawn with replacement from the original dataset) and training a separate model on each of these datasets. The key idea is that by combining predictions from models trained on slightly different datasets, the overall variance of the model decreases, leading to more accurate and reliable predictions.

Steps of Bagging:

1. **Data Sampling**: Multiple subsets of the original dataset are created using bootstrapping (random sampling with replacement). Each subset may contain duplicate instances but will be different from the others.
2. **Model Training**: A separate model is trained on each bootstrapped subset of the data. These models are typically of the same type, such as decision trees, but each model sees different data.
3. **Aggregation**: For classification tasks, the final prediction is made by taking a majority vote across all models. For regression tasks, the final prediction is the average of all models' predictions.

Advantages of Bagging:

1. **Reduces Variance**: By combining multiple models, Bagging reduces the risk of overfitting to any particular subset of the data.
2. **Improves Stability**: Models trained on bootstrapped datasets are less likely to be influenced by outliers or noise in the data.
3. **Parallelization**: Since each model is trained independently, Bagging allows for easy parallelization, which can speed up training when resources are available.
4.

Mathematical equation :

$$\hat{y} = 1/M \sum(m = 1)^{M} h_m(x)$$

This is the mathematical foundation of Bagging. By averaging multiple predictions, we cancel out individual errors (noise), which significantly reduces the Variance of the model. This is exactly how a Random Forest calculates its final numerical output.

Algorithm: Random Forest

One of the most well-known implementations of Bagging is the **Random Forest** algorithm. In Random Forests, multiple decision trees are trained on different bootstrapped samples of the data, and the predictions of the trees are aggregated. This method is particularly effective because it combines the strengths of decision trees with the stability and robustness of Bagging.

Use Case: Random Forests are commonly used in tasks like classification (e.g., predicting whether a customer will churn) and regression (e.g., predicting house prices) due to their ability to handle large datasets, categorical variables, and complex relationships in the data.

Random Forest is a "team-based" learning method. It builds many individual **Decision Trees** and combines their results to get a more accurate and stable answer. It uses two main tricks: **Bagging** and **Feature Randomness**.

The Core Trick: Bagging (Data Sampling)

Instead of giving every tree the exact same data, we give each tree a slightly different "view" of the data.

- **The Math:** From a dataset of size **N**, we create many samples. We pick data points at random **with replacement**.

- **The Result:** About **63%** of the data in each sample is unique, and **37%** is left out (called Out-of-Bag data). This helps the model stay flexible and not memorize the data.

The Forest Trick: Feature Randomness

In a normal tree, the computer looks at **every** category (feature) to find the best split. In a Random Forest, each tree is forced to only look at a **random subset** of categories.

- **For Classification:** Usually, the tree looks at the square root of the total features (√p).
- **For Regression:** Usually, it looks at one-third of the total features (p/3).
- **Why?** This keeps the trees from becoming too similar. If one feature is very strong, this trick forces some trees to ignore it and find other hidden patterns.

3. How the Final Answer is Calculated

Once all the trees (let's call the total number of trees **B**) have made their individual guesses, the "Forest" combines them:

- **For Numbers (Regression):** The final answer is the **Average** of all trees.
  Formula: Final Prediction = (1 / Total Trees) × Sum of all tree guesses.
- **For Categories (Classification):** The final answer is the **Majority Vote**.
  Formula: Final Prediction = The category that received the most votes from the trees.

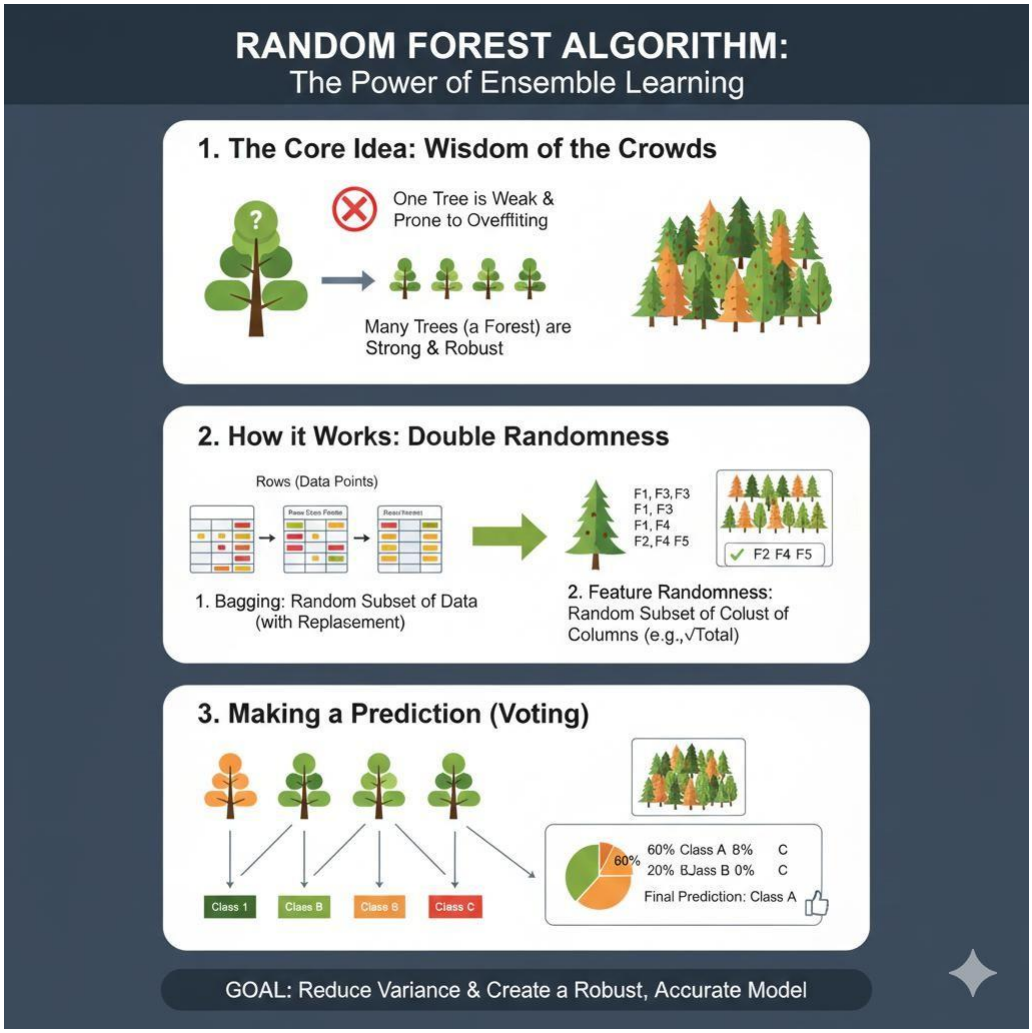

Figure 14

5.2 What is Boosting?

Boosting is another powerful ensemble technique in machine learning, but unlike Bagging, Boosting focuses on **reducing bias** rather than variance. The main idea behind Boosting is to train models sequentially, where each new model tries to correct the errors made by the previous models. Boosting creates a strong model by combining the predictions of weak learners (models that are only slightly better than random guessing), improving accuracy and performance over time.

How Boosting Works

Boosting works by building models one at a time, with each subsequent model focusing on the mistakes made by the previous models. The key to Boosting's success lies in its ability to learn from past mistakes and improve gradually.

Steps of Boosting:

1. **Model Initialization**: The process starts with training a weak learner (usually a simple model like a decision tree) on the entire dataset.
2. **Weight Adjustment**: The algorithm assigns higher weights to the incorrectly classified examples, emphasizing them in the next round of training. This ensures that the subsequent model focuses on these difficult examples.
3. **Model Combination**: Each model is added to the ensemble, and its prediction is combined with those of the previous models. In the case of classification, a weighted majority vote is often used, while for regression, the models' predictions are averaged.

Popular boosting algorithm

1. AdaBoost (Adaptive Boosting)

Sequentially trains models by increasing the weights of **misclassified data points** from the previous round. It then combines all models using a weighted vote based on their accuracy.Simple binary classification (e.g., Spam or Fraud detection).Very effective at focusing on "hard to classify" cases.

2. Gradient Boosting Machines (GBM)

A generalization of AdaBoost that uses **Gradient Descent**. Instead of re-weighting points, it trains the next model to predict the **residual errors** (the gap between actual and predicted values) of the current ensemble.General predictive analytics (e.g., Credit risk, Churn prediction).**Pros:** Highly accurate for both regression and classification.

3. XGBoost (Extreme Gradient Boosting)

A highly optimized version of GBM. It adds **Regularization** (to prevent overfitting) and supports **Parallel Processing** for massive speed gains.Data science competitions and production-level time-series forecasting.The "gold standard" for performance and speed.

4. LightGBM

Designed by Microsoft for massive datasets. It uses **Leaf-wise tree growth** (growing the most "error-reducing" nodes first) and histogram-based learning to save memory.Large-scale industrial tasks

where training time and memory are limited.Significantly faster and more memory-efficient than standard XGBoost.

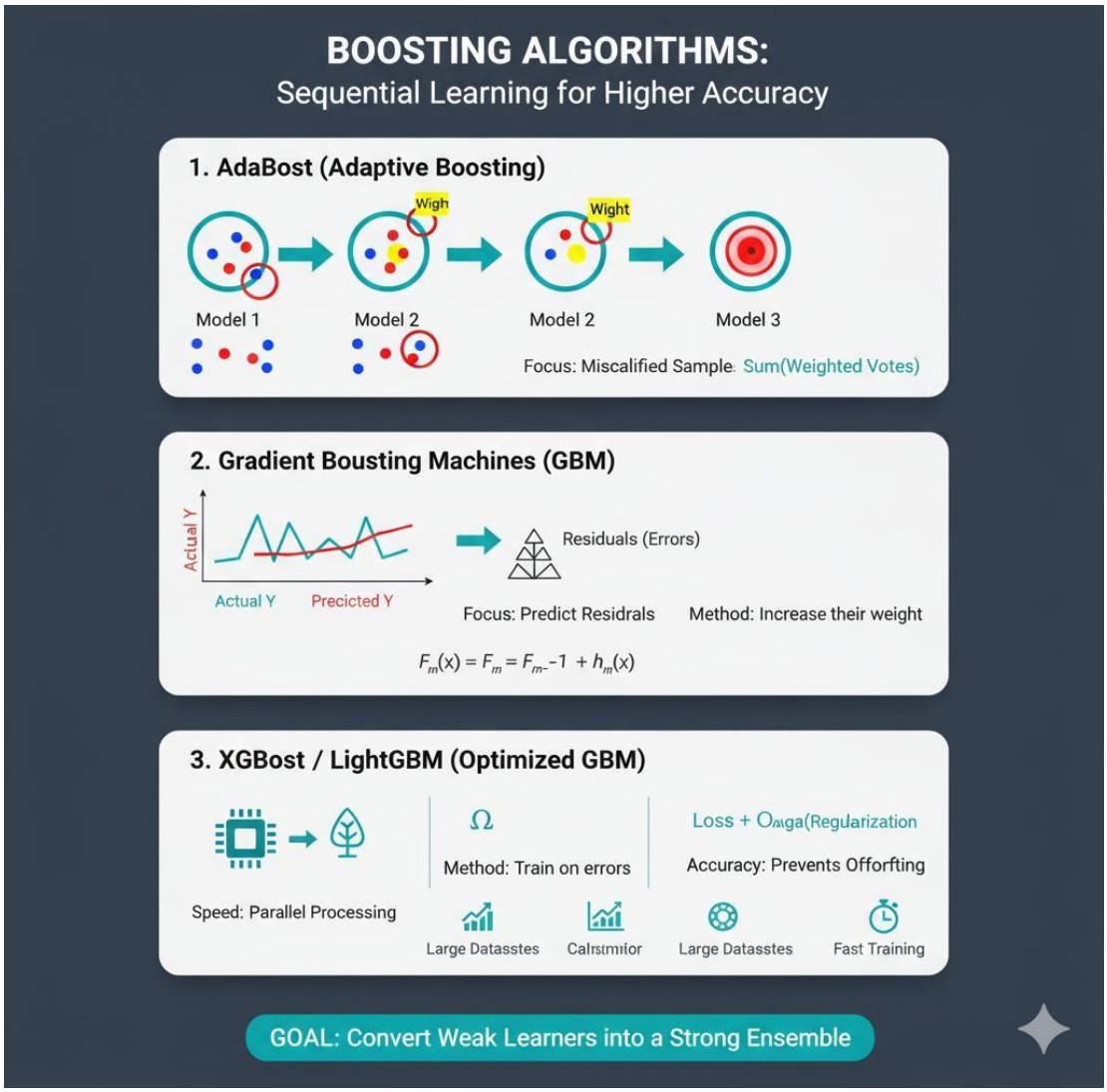

Figure 15

Difference between bagging and boosting

Feature	Bagging	Boosting
Purpose	Reduces **variance** in high-variance models.	Reduces **bias** by sequentially correcting errors.
Model Independence	Models are trained **independently** in parallel.	Models are **dependent**; each corrects the last.
Weighting of Models	**Equal weight** given to all models.	Weights adjusted based on **performance**.
Training Data	Random subsets with replacement (Bootstrap).	Focuses on data **misclassified** by previous models.
Iterative Process	**No**; models do not depend on one another.	**Yes**; iterative improvement of errors.
Combining Logic	Majority voting or simple averaging.	Weighted combination based on accuracy.
Parallelism	Supports **parallel processing** (faster).	Primarily **sequential** processing.
Best For	Deep trees and noisy data.	Shallow trees and high-bias datasets.
Popular Example	**Random Forest**	**AdaBoost, XGBoost, LightGBM**

Table 9

Key points to remember

1. **Classification predicts discrete class labels**, while **regression predicts continuous numerical values**.
2. Classification answers *"Which class?"* whereas regression answers *"How much?"*.
3. Classification problems use **decision boundaries**, regression problems use **best-fit functions**.
4. Classification errors are measured as **misclassification**, regression errors as **prediction distance**.
5. Classification is evaluated using **confusion-matrix metrics**, regression using **error-based metrics**.
6. **Binary classification** assigns data into exactly two classes (Yes/No).
7. **Multi-class classification** assigns one label from more than two possible classes.
8. **Multi-label classification** allows an instance to belong to multiple classes simultaneously.
9. **Nominal classification** uses unordered categorical labels.
10. **Ordinal classification** uses ordered categories without numeric distance.
11. **C4.5 uses Gain Ratio** to avoid bias toward multi-valued attributes.
12. **CART always produces binary trees**, regardless of feature type.
13. **C4.5 uses Error-Based Pruning**, while **CART uses Cost-Complexity Pruning**.
14. **Gini Index measures node impurity**, entropy measures uncertainty.
15. **Naive Bayes assumes conditional independence** among features given the class.
16. **Bayesian Networks model dependencies explicitly** using a Directed Acyclic Graph (DAG).
17. Naive Bayes is computationally fast because it converts joint probability into a **product of marginals**.
18. **Accuracy is unreliable for imbalanced datasets**.
19. **Precision focuses on false positives**, while **Recall focuses on false negatives**.
20. **F1-score balances Precision and Recall using harmonic mean**.
21. **MSE penalizes large errors more heavily due to squaring**.
22. **RMSE brings error back to the original unit of the target variable**.
23. **R^2 measures explained variance**, not prediction error.
24. **Bagging reduces variance by averaging independent models trained on bootstrapped data**.
25. **Boosting reduces bias by sequentially correcting previous model errors**.
26. **Random Forest combines Bagging and feature randomness to reduce variance and correlation**.
27. **High bias causes underfitting**, **high variance causes overfitting**.
28. **Total error equals $Bias^2$ + Variance + Irreducible Error**.
29. **Linear regression assumes a linear relationship between predictors and response**.
30. **Polynomial regression captures nonlinear relationships using higher-degree terms**.
31. **Lasso (L1) performs feature selection by shrinking coefficients to zero**.
32. **Ridge (L2) reduces coefficient magnitude but keeps all features**.
33. **Elastic Net combines L1 and L2 to handle collinearity and sparsity together**.

-- Practice Questions --

A. MCQs (Multiple Choice Questions)

1. Which of the following best describes **classification**?
   a) Predicting numerical values
   b) Predicting continuous outcomes
   c) Assigning data to predefined categories
   d) Finding correlation
   Ans: c
2. Which metric is best suited for evaluating **imbalanced classification datasets**?
   a) Accuracy
   b) Precision
   c) Recall
   d) F1-Score
   Ans: d
3. In Naive Bayes, the **independence assumption** means:
   a) Classes are independent
   b) Features are conditionally independent given the class
   c) Samples are independent
   d) Output is independent
   Ans: b
4. Which algorithm uses **Gain Ratio** as a splitting criterion?
   a) CART
   b) ID3
   c) C4.5
   d) Random Forest
   Ans: c
5. Gini Index is used in:
   a) C4.5
   b) CART
   c) Naive Bayes
   d) Logistic Regression
   Ans: b
6. Which regression metric penalizes **large errors more heavily**?
   a) MAE
   b) RMSE
   c) MSE
   d) R^2
   Ans: c
7. Boosting mainly helps in reducing:
   a) Variance
   b) Bias
   c) Noise
   d) Dimensionality
   Ans: b

8. Random Forest reduces variance mainly by:
   a) Sequential learning
   b) Feature elimination
   c) Bootstrap sampling and feature randomness
   d) Gradient descent
   Ans: c
9. Which algorithm finds an optimal hyperplane?
   a) k-NN
   b) Decision Tree
   c) SVM
   d) Naive Bayes
   Ans: c
10. Which regression technique performs **feature selection**?
   a) Ridge
   b) Lasso
   c) Linear Regression
   d) Polynomial Regression
   Ans: b

B. SQs (Short Questions)

1. Define classification and regression.
2. What is the difference between binary and multi-class classification?
3. State Bayes' Theorem used in Naive Bayes.
4. What is the Gini Index?
5. Define Precision and Recall.
6. What is Mean Squared Error (MSE)?
7. Explain the bias–variance tradeoff.
8. What is bootstrapping in Bagging?
9. What is feature randomness in Random Forest?
10. Define ordinal classification with an example.

C. LQs (Long Questions)

1. Explain classification and regression with suitable examples and a comparison table.
2. Discuss types of classification techniques in data mining with mathematical representation.
3. Explain the C4.5 algorithm with Gain Ratio and pruning.
4. Describe the CART algorithm and compare it with C4.5.
5. Explain Naive Bayes classifier with assumptions, formula, and numerical example.
6. Explain Bayesian Networks with DAG and joint probability formulation.
7. Discuss classification evaluation metrics using a confusion matrix.
8. Explain regression evaluation metrics (MSE, RMSE, MAE, R^2, Adjusted R^2).
9. Explain ensemble learning and discuss Bagging with Random Forest.
10. Compare Bagging and Boosting with advantages, limitations, and examples.